

详细设计篇期末复习

依据：课程 PPT、123.txt 答疑转写、语音版重点、考试题目类型与重点；往年卷用于例题风格参考。

使用建议

详细设计题的核心不是背一个模式名，而是把需求和体系结构继续落到类、对象协作和可修改性设计上。复习时按“职责分配 - 类图 - 详细顺序图 - 设计原则 - 变化点封装”的顺序练。

一、考试定位与优先级

优先级	内容	为什么重要	答题产物
高	GRASP : Information Expert、Creator、Controller	老师口头重点明确提到；详细设计要解决职责分配	说明谁负责什么、为什么由它负责，并画类图/顺序图
高	类的设计原则：OCP、DIP、SRP、LSP、ISP、组合优于继承等	答疑说类的原则会考，可能组合考多个	判断违反原则、说明原因、给出重构方案
高	详细顺序图	2022 往年卷出现“对象交互顺序图”	Actor/Boundary/Controller/Service/DAO/Entity 的对象协作
中高	设计模式诱导题：策略模式、工厂/抽象工厂、状态模式等	今年设计模式更偏诱导式，不单纯背模式名	找到变化点，用接口+多态封装变化
中	状态图与代码转换、控制风格	答疑强调状态图主体；课件有集中/委托/分散控制风格	明确状态主体、状态迁移，或说明控制职责分配
中	内聚耦合，尤其控制耦合和印记耦合	不是最重，但答疑专门展开提醒	识别 flag 参数、过量参数对象，提出多态/只传必要参数

一句话判断

详细设计题通常在问：这个系统内部应该有哪些类？每个类负责什么？对象之间按什么顺序协作？当规则、优惠券、支付方式变化时，怎样少改旧代码？

二、详细设计基础

1. 什么是详细设计

详细设计是把体系结构中的模块继续细化到可编码粒度的设计活动。体系结构回答“系统有哪些大模块、怎么连接”，详细设计回答“模块内部有哪些类、方法、对象协作和算法”。

维度	来自需求/体系结构的输入	详细设计输出
静态结构	概念类图、模块接口、PO/VO/DTO	设计类图：类、接口、属性、方法、继承/实现/关联/组合
动态行为	用例、系统顺序图、业务流程	详细顺序图/通信图：对象之间如何发消息
状态行为	状态图、复杂对象生命周期	状态机实现、State 模式或 switch/transition 设计
质量要求	可修改性、性能、安全、可测试性	设计理由：为什么这样分配职责、如何降低耦合

详细设计的输入

来自需求工程 (RE)

- 用例 (Use Case) —— 功能场景
- 领域模型 / 概念类图 —— 业务实体
- 系统顺序图 —— 系统级交互
- 状态图 —— 复杂对象生命周期
- 非功能需求 —— 性能、可靠性、安全等

来自体系结构

- 模块的规格 (Specification)
- 导入/导出接口 (Export / Import)
- 风格约束 (分层、MVC、微服务.....)
- 构件间接口 (如 SalesBLService、SalesDataService)

桥接类比：需求规格说明 ("在两座山间建一座桥，5 分钟内 100km/h 通过") + 体系结构 ("斜拉桥风格：桥墩、桥身、悬索") → 详细设计 ("悬索：拉索的根数；路面：路面材质")。

SWEBOK v4 §1.2 · Context of Software Design | §2.2 · Detailed Design

9

课件截图：详细设计输入。需求、系统顺序图、概念类图和体系结构接口都会影响详细设计。

详细设计的输出

视角	产出物	UML 表示
静态结构	模块内部的类、接口与关系	类图、包图
动态行为	类之间的协作与消息传递	顺序图、通讯图
状态行为	复杂对象的生命周期	状态图
实现注解	构造方法、枚举、常量、静态方法说明	文字 + 代码片段
质量论证	为何这样设计：耦合/内聚/扩展性	设计理由 (Rationale)

详细设计输出 ≠ "画几张图"，而是为后继开发者提供足够、清晰、可论证的实现蓝图。

SWEBOK v4 §4 · Recording Software Designs | §4.2 · Structural Design Descriptions | §4.3 · Behavioral Design Descriptions | §4.6 · Design Rationale

11

课件截图：详细设计输出。注意详细设计不是只画图，还要能说明设计理由。

2. 概念类图和设计类图的区别

对比	概念类图	设计类图
阶段	需求分析/领域建模	详细设计
类的含义	问题域概念，例如产品、库存、订单	软件实现类，例如 Service、Controller、Repository、策略类
内容	类名、重要属性、概念关系；不含方法	类名、属性、方法、接口、可见性、实现关系
重点	理解业务对象和关系	分配职责、支撑代码实现和可修改性

GRASP Patterns

- GRASP = General Responsibility Assignment Software Patterns
- Not 'design patterns', rather **fundamental principles** of object design
- Focus on one of the most important aspects of object design: **assigning responsibilities to classes**

5 个核心模式（本讲覆盖）

#	模式	解决的问题
1	Low Coupling	降低模块之间的耦合
2	High Cohesion	保持模块职责的聚焦
3	Information Expert	把职责分配给“拥有信息”的类
4	Creator	由谁负责创建对象
5	Controller	由谁负责处理系统事件

Larman 著《UML 和模式应用》是 OO 详设的经典参考。

SWEBOK v4 §5.4 · Object-Oriented Design | §1.4 · Software Design Principles (Coupling, Cohesion)

25

课件截图：详细设计中的类关系。关联、聚合、组合、继承、实现都可能进入设计类图。

三、面向对象详细设计：职责与协作

1. 职责 Responsibility

- 职责就是一个类应该知道什么、维护什么数据，或者应该执行什么任务。
- 数据职责通常变成属性，操作职责通常变成方法。
- 详细设计的本质，是把系统职责逐步分配给具体类，并让这些类通过消息协作完成用例。

2. 协作 Collaboration

- 单个对象通常不能独立完成完整用例，需要和其他对象协作。
- 协作在详细顺序图中体现：谁先收到消息，谁调用谁，谁返回结果。
- 好的协作应该让每个对象做自己最擅长、最有信息的事情，避免一个 God Class 做所有事。

答题模板

先说明系统操作由 Controller 接收；Controller 不直接做所有业务，而是委托给 Service/领域对象；拥有数据的类承担计算职责；数据访问交给 DAO/Repository；变化点通过接口、多态或策略类封装。

四、GRASP 模式

1. 总览

GRASP	解决的问题	考试答法
Information Expert 信息专家	某个职责应该给谁？	给拥有完成该职责所需信息的类
Creator 创建者	谁负责创建某对象？	由包含/聚合/记录/紧密使用该对象，或拥有初始化数据的类创建
Controller 控制器	谁接收系统事件？	由系统、子系统、用例控制器或外观控制器接收，再协调其他对象
Low Coupling 低耦合	如何减少类之间依赖？	避免不必要依赖，依赖接口，职责分配不引入过多连接
High Cohesion 高内聚	如何让类职责集中？	每个类围绕清晰职责，不把无关任务塞到同一类

GRASP Patterns

- GRASP = General Responsibility Assignment Software Patterns
- Not 'design patterns', rather **fundamental principles** of object design
- Focus on one of the most important aspects of object design: **assigning responsibilities to classes**

5 个核心模式（本讲覆盖）

#	模式	解决的问题
1	Low Coupling	降低模块之间的耦合
2	High Cohesion	保持模块职责的聚焦
3	Information Expert	把职责分配给“拥有信息”的类
4	Creator	由谁负责创建对象
5	Controller	由谁负责处理系统事件

Larman 著《UML 和模式应用》是 OO 设计的经典参考。

SWEBOK v4 §5.4 · Object-Oriented Design | §1.4 · Software Design Principles (Coupling, Cohesion)

25

课件截图：GRASP 总览。考试中重点用它解释“为什么由这个类负责”。

2. Information Expert 信息专家

把职责分配给拥有完成该职责所需信息的类。例如计算订单总价，应由订单或订单项相关类负责，而不是界面类负责；计算库存是否低于阈值，应由库存/规则相关对象或服务协调完成。

③ Information Expert（信息专家）

- **Problem**: What is the most basic principle by which responsibilities are assigned in object-oriented design?
- **Solution**: **Assign a responsibility to the class that has the information necessary to fulfill the responsibility.**

案例

要计算 `Sales.total()`：

- 谁知道每个 `SalesLineItem` 的数量？→ `SalesLineItem`
- 谁知道每个 `Commodity` 的价格？→ `Commodity`
- → `Sales.total()` 委托给 `SalesLineItem.subtotal()`，后者再调 `Commodity.getPrice()`

"谁有数据，谁负责" —— 自然形成职责链，避免数据外泄。

28

课件截图：信息专家。判断标准不是“哪个类名字像”，而是谁拥有所需信息。

3. Creator 创建者

- B 聚合或组合 A，则 B 常负责创建 A。
- B 记录、紧密使用 A，或拥有初始化 A 的数据，也可以由 B 创建 A。
- 如果创建逻辑复杂或需要隔离变化，可以引入 Factory，而不是把创建散落各处。

Collaboration (Control) Styles

A control style is a way that all system behavior is distributed among objects (components) network.

风格	特征	控制复杂度
Centralized (集中式)	A few controllers record logics of all system behaviors	控制器高复杂度，其他对象低复杂度
Delegated (委托式)	Decision making is distributed through the object networks with a few controllers making the main decisions	中复杂度对象有几个，其他低复杂度
Dispersed (分散式)	All system behavior is spread widely through the objects network	所有对象都低复杂度，但交互密集

34

课件截图：Creator。单据创建明细项、上下文创建策略对象，都是常见考点。

4. Controller 控制器

- Controller 负责接收来自 UI/API 的系统事件，例如 `recharge(...)`、`runSchedule(...)`。
- 它应协调对象，而不是承担所有业务计算。业务规则应下放给 Service/领域对象。
- 可选控制器：系统整体、子系统、设备/组织外观、用例控制器、人工 Pure Fabrication 控制器。

⑤ Controller (控制器)

- **Problem**: How to assign responsibility for handling a system event?
- **Solution**: If a program receives events from external sources other than its graphical interface, add an event class to **decouple the event source(s) from the objects that actually handle the events.**

4 种可选的控制器

1. **The business or overall organization** (façade controller) — e.g. `Store`
2. **The overall "system"** (façade controller) — e.g. `POST`
3. **An animate thing in the domain** that would perform the work (role controller) — e.g. `Cashier`
4. **An artificial class** (Pure Fabrication) representing the use case (use case controller) — e.g. `BuyItemsHandler`

SWEBOK v4 §5.4 · Object-Oriented Design | §3.2 · Control and Event Handling

31

课件截图：Controller。控制器连接界面事件和领域对象协作。

Parnas · 层次结构与封装

- **Hierarchical Structure**
 - 模块或程序之间可定义某种关系
 - 该关系是偏序
 - 我们关心的关系是 "use" or "depends upon"
- 数据结构、其内部链接、访问过程与修改过程都是单一模块的一部分 —— 即封装 (encapsulation) 的思想
- 控制块的格式必须隐藏在 "control block module" 内部

模块化 ≠ 把代码切碎。模块化的本质是：让每个模块封装一个“会改变的设计决策”—— 当那个决策改变时，只需修改一个模块。

43

课件截图：控制器的坏设计与好设计。避免表现层直接操纵领域对象，也避免控制器膨胀。

五、详细顺序图

1. 系统顺序图 vs 详细顺序图

对比	系统顺序图	详细顺序图
视角	系统作为黑盒	系统内部对象协作
生命线	Actor、System	Actor、Boundary、Controller、Service、DAO/Repository、Entity
来源	用例主流程/系统级需求	系统顺序图中的某个系统操作 + 体系结构接口 + 领域模型
考试信号	“参与者与系统交互”	“对象交互” “类之间协作” “充值时对象交互”

2. 绘制步骤

- 1 选定一个系统操作，例如 `recharge(cardId, amount, payMethod)` 或 `runSchedule()`。
- 2 找 Boundary：页面/API/界面对象，负责接收用户输入。
- 3 找 Controller：接收系统事件并协调流程。
- 4 找 Service/领域对象：执行业务规则，如计算优惠、判断库存、生成单据。
- 5 找 DAO/Repository：查询和保存持久化对象。
- 6 按时间顺序画同步调用和返回消息；有分支时使用 `alt`。

```

Actor -> Boundary: submit(...)
Boundary -> Controller: operation(...)
Controller -> Service: operation(...)
Service -> Repository: find/save/update(...)
Repository --> Service: PO/entity
Service -> DomainObject/Strategy: calculate(...)
Service --> Controller: ResultVO
Controller --> Boundary: show(result)
Boundary --> Actor: display result

```

UML 箭头规范

同步消息：实线 + 实心三角箭头；异步消息：按老师答疑记为虚线 + 鱼骨箭头；返回消息：虚线 + 开放箭头。考试手绘时要区分“调用”和“返回”。

六、控制风格与状态图

1. 控制风格

控制风格	含义	优缺点/适用
集中式	一个 Controller/FSM 统一发起调用	流程清楚，容易调试；但控制器容易膨胀
委托式	控制器保留主决策，将子任务委托给专家对象	更符合高内聚低耦合，是考试中较稳的设计
分散式	多个对象各自承担局部决策	扩展灵活，但对象交互复杂，理解成本高

Collaboration (Control) Styles

A control style is a way that all system behavior is distributed among objects (components) network.

风格	特征	控制复杂度
Centralized (集中式)	A few controllers record logics of all system behaviors	控制器高复杂度，其他对象低复杂度
Delegated (委托式)	Decision making is distributed through the object networks with a few controllers making the main decisions	中复杂度对象有几个，其他低复杂度
Dispersed (分散式)	All system behavior is spread widely through the objects network	所有对象都低复杂度，但交互密集

34

课件截图：控制风格。答题时优先解释为什么避免控制器过胖，为什么委托给信息专家。

2. 状态图

- 画状态图前先确定主体：是单个订单、一张充值记录、一个红绿灯，还是整个路口系统。
- 状态图表达某个对象/主体在生命周期中的状态和迁移，不是画所有类之间的调用。
- 状态图可以转为代码：简单情况用 switch/if；复杂且状态行为变化明显时可用 State 模式。

状态图概念 [SWEBOK 4.4]

- 描述系统、用例或对象在**整个生命期内的状态变化**
- 适用场景：订单状态流转、设备状态、用户账户状态等
- 也称为"状态机图" (State Machine Diagram)

基本元素

元素	符号	说明
状态 (State)	圆角矩形	对象在某一时刻的条件或情况
转换 (Transition)	箭头	从一个状态到另一个状态的变迁
事件 (Event)	转换上的标签	触发状态转换的外部刺激
动作 (Action)	/ action	状态转换时执行的操作
初始状态	实心圆	状态机的起点
终止状态	圆中圆	状态机的终点

38

课件截图：状态图概念。答疑里老师特别强调“先明确主体”。

状态图 → Java 代码的映射 [SWEBOK 4.4]

映射规则

状态图元素	Java 对应
状态	枚举值
转换	方法
守卫条件	if 检查
非法转换	抛出异常
动作	方法内逻辑

状态图中的每个转换 → 一个方法；每个守卫条件 → 方法开头的 if 检查

代码示例

```
public enum OrderStatus {
    PENDING_PAYMENT, PAID, SHIPPED,
    COMPLETED, CANCELLED, REFUNDING
}

// 待支付 → 已支付
public void pay() {
    if (status != PENDING_PAYMENT)
        throw new IllegalStateException();
    status = PAID;
    deductInventory(); // 动作：扣减库存
}

// 已支付 → 已发货
public void ship(String trackingNo) {
    if (status != PAID)
        throw new IllegalStateException();
    status = SHIPPED;
}

// 多条 → 已取消 (守卫条件不同)
public void cancel() {
    if (status == PENDING_PAYMENT) {
        status = CANCELLED;
        releaseInventory(); // 释放库存
    } else if (status == PAID) {
        status = CANCELLED;
        releaseInventory();
        initiateRefund(); // 发起退款
    } else throw new IllegalStateException();
}
```

43

课件截图：状态图到代码。复杂状态行为可转成 State 模式。

七、类的设计原则

1. 必背原则表

原则	核心含义	常见违反	修改方向
SRP 单一职责	一个类只有一个变化原因	User 类同时负责数据、支付、发券、导出报表	拆分类，把不同变化原因分离
OCP 开闭原则	对扩展开放，对修改关闭	新增优惠券/支付方式要改原有 switch	抽象接口 + 新增实现类 + 多态

原则	核心含义	常见违反	修改方向
DIP 依赖倒置	高层模块依赖抽象，不依赖具体实现	Service 直接 new WeChatPay	依赖 PaymentStrategy/Repository 接口
ISP 接口隔离	客户端不依赖不使用的方法	Robot 被迫实现 eat/sleep	拆成多个小接口
LSP 里氏替换	子类能替换父类且不破坏契约	企鹅继承 Bird 但 fly 抛异常	重新抽象父类或拆接口
LoD 迪米特法则	只与你的直接朋友交谈	a.getB().getC().doX()	通过委托封装消息链
组合优于继承	优先用组合进行黑盒复用	为复用代码滥用继承	持有接口对象，运行时可替换
权限最小化	从 private 开始，必要时再放宽	public 字段、暴露集合内部	封装字段，提供受控方法

原则汇总（P1—P7）：信息隐藏基础 · 访问耦合 · 继承耦合

完整编号体系：P1—P4 见 04—01（信息隐藏基础），P5—P14 见本讲（面向对象设计）

编号	缩写	原则	核心思想	SWEBOK
P1	—	全局变量有害	全局变量穿透所有模块，是隐藏的反义词	§1.4 Encapsulation
P2	—	优先显示表达	用接口/文档显式表达约束，不靠隐式约定	§1.4
P3	DRY	不要重复	同一决策不在多处编码；重复=多处修改	§1.4
P4	—	面向接口编程	依赖接口而非实现，降低访问耦合	§1.4 Sep. of I&I; §5.4
P5	LoD	迪米特法则	只与直接朋友交谈，避免消息链	§5.4 OOD
P6	ISP	接口隔离	接口细化，客户端只依赖需要的方法	§5.4 SOLID-I
P7	LSP	里氏替换	子类前置放宽、后置加强、不变式维持	§5.4 SOLID-L

37

课件截图：原则 P1-P7。类原则可能组合考，重点会让你判断违反了什么原则。

原则汇总（P8—P14）：封装变更 · SOLID · SOFA

编号	缩写	原则	核心思想	SWEBOK
P8	—	组合优于继承	黑盒复用，避免继承耦合	§5.4 GoF
P9	—	为变更而封装	隔离可变部分，稳定不变部分	§1.4 Encapsulation
P10	—	权限最小化	从 <code>private</code> 开始，按需放宽	§1.4
P11	OCP	开闭原则	对扩展开放，对修改关闭	§5.4 SOLID-O
P12	DIP	依赖倒置	高层依赖抽象，不依赖具体实现	§5.4 SOLID-D
P13	SRP	单一职责	一个类只有一个变化原因	§5.4 SOLID-S
P14	SOFA	方法设计	Short · One thing · Few args · Abstraction	§5.4 SOFA

记忆结构：P1—P4 防止"隐式依赖"；P5—P8 控制"耦合来源"；P9—P13 管理"变化封装"；P14 约束"方法粒度"。

SWEBOK v4 §1.4 · Software Design Principles / §5.4 · Object-Oriented Design (SOLID)

38

课件截图：原则 P8-P14。OCP、DIP、SRP、组合优于继承是变化题里的常用答案。

2. 内聚与耦合

- 高内聚：一个类/方法内部元素围绕同一职责，修改理由集中。
- 低耦合：类之间依赖少、依赖稳定抽象、少暴露实现细节。
- 控制耦合：调用者传入 flag/type，方法内部 switch 决定行为。修改方向：多态、策略模式、命令对象。
- 印记耦合：把一个大对象传给方法，但方法只需要其中一两个字段。修改方向：只传必要参数或提取专用参数对象。

```
// ■■■■■type ■■■■■
void pay(String type, Money amount) {
    if (type.equals("WECHAT")) { ... }
    else if (type.equals("ALIPAY")) { ... }
}

// ■■■■■ + ■■
interface PaymentStrategy { PayResult pay(Money amount); }
class WeChatPay implements PaymentStrategy { ... }
class AliPay implements PaymentStrategy { ... }
```

P6 · 接口隔离原则 (ISP)

Clients should not be forced to depend on interfaces they do not use.

肥接口问题

```
interface Worker {
    void work();
    void eat();    // Robot 不需要!
    void sleep(); // Robot 不需要!
}

class Robot implements Worker {
    public void work() { ... }
    public void eat() { throw new UnsupportedOperationException(); }
    public void sleep() { throw new UnsupportedOperationException(); }
}
```

ISP 重构

```
interface Workable {
    void work();
}

interface Feedable {
    void eat();
    void sleep();
}

class Robot implements Workable { ... }
class Human implements Workable,
    Feedable { ... }
```

接口细化后，Robot 不再被迫实现无关方法，调用方只依赖自己真正需要的接口。

课件截图：接口隔离原则。接口越胖，调用方被迫知道/实现越多无关内容。

P11 · 开闭原则 (OCP)

Software entities should be open for extension, but closed for modification.
— Bertrand Meyer

违反 OCP 的典型模式

```
// 每次增加新类型都要修改
class DiscountCalculator {
    double calculate(Order order) {
        if (order.type == VIPORDER)
            return order.total * 0.8;
        else if (order.type == NEWUSER)
            return order.total * 0.9;
        // 增加新类型需修改本类...
    }
}
```

符合 OCP 的重构

```
interface DiscountPolicy {
    double apply(double total);
}

class VipDiscount implements DiscountPolicy {
    public double apply(double t) {
        return t * 0.8;
    }
}

class DiscountCalculator {
    private DiscountPolicy policy;
    // 新增类型只需新增实现类
    double calculate(Order o) {
        return policy.apply(o.getTotal());
    }
}
```

33

课件截图：组合优于继承。策略模式、支付方式、优惠券规则都常用组合和多态。

八、设计模式诱导题

1. 答题套路

- 1 先找变化点：优惠券类型、支付方式、订货规则、状态行为、对象创建族。
- 2 判断旧设计问题：switch/if 扩散、控制耦合、违反 OCP/DIP/SRP。
- 3 抽象稳定接口：如 `CouponStrategy`、`PaymentStrategy`、`ReorderRule`。
- 4 把每种变化做成实现类：新增类型只新增类，不改旧业务流程。
- 5 上下文类组合接口对象，并在运行时选择或注入具体实现。
- 6 画类图：Context 持有 Strategy 接口，ConcreteStrategy 实现接口。

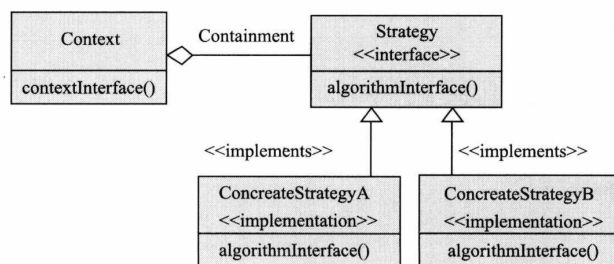


图 16-8 策略模式

策略模式：定义了算法族，分别封装起来，让他们之间可以互相替换，此模式让算法的变化独立于使用算法的客户。

策略模式类图

课件截图：策略模式。遇到“未来还会新增多种算法/优惠/支付方式”，优先想到它。

协作

- 上下文Context和Strategy的相互协作完成整个算法。Context可能会通过提供方法让Strategy访问其数据；甚至将自身的引用传给Strategy，供其访问其数据。Strategy会在需要的时候访问Context的成员变量。
- 上下文Context将一些对他的请求转发给策略类来实现，客户（Client）通常创建ConcreteStrategy的对象，然后传递给Context来灵活配置Strategy接口的具体实现；这样Client就有可以拥有一个Strategy接口的策略族，其中包含多种ConcreteStrategy的实现。

课件截图：策略模式类图。核心是 Context 组合 Strategy，具体策略通过多态替换。

九、往年卷风格例题与答案

例题 1：2022 校园卡充值 - 优惠券和支付方式

题型：用户充值时有不同优惠券，同时可以用支付宝、微信等不同充值方式。要求画 User、Card、多种 Coupon、ThirdPayment、PaymentRecord 类图；新增数值折扣、比例折扣如何实现变更；画充值对象交互顺序图。

1. 类图答案思路

```
User 1 ---- 1 Card
User 1 ---- 0..* Coupon
Card 1 ---- 0..* PaymentRecord

<<interface>> Coupon
+discount(amount): Money
FixedAmountCoupon implements Coupon
PercentageCoupon implements Coupon

<<interface>> ThirdPayment
+pay(userId, amount): PayResult
WeChatPayment implements ThirdPayment
AliPayPayment implements ThirdPayment

RechargeService
- payment: ThirdPayment
+recharge(user, card, amount, coupon): RechargeResult
```

- 优惠券变化点用 `Coupon` 接口封装，固定金额优惠券和比例优惠券分别实现。
- 支付方式变化点用 `ThirdPayment` 接口封装，微信、支付宝分别实现。
- RechargeService 依赖抽象接口，符合 DIP；新增优惠券或支付方式时新增类，符合 OCP。
- User 和 Card 是领域对象，PaymentRecord 记录充值结果，Card 与 PaymentRecord 可画 1 对 0..* 关联。

2. 充值详细顺序图答案模板

```
User -> RechargePage: submit(amount, couponId, payType)
RechargePage -> RechargeController: recharge(requestV0)
RechargeController -> RechargeService: recharge(requestV0)
RechargeService -> CouponRepository: find(couponId)
CouponRepository --> RechargeService: Coupon
RechargeService -> Coupon: discount(amount)
Coupon --> RechargeService: discountedAmount
RechargeService -> PaymentFactory: getPayment(payType)
PaymentFactory --> RechargeService: ThirdPayment
RechargeService -> ThirdPayment: pay(userId, discountedAmount)
ThirdPayment --> RechargeService: PayResult
alt
    RechargeService -> CardRepository: updateBalance(cardId, discountedAmount)
    RechargeService -> PaymentRecordRepository: save(record)
    RechargeService --> RechargeController: RechargeResultV0(success)
else
    RechargeService --> RechargeController: RechargeResultV0(failReason)
end
RechargeController --> RechargePage: show(result)
```

例题 2：2020 进销存 - 再订货规则变化

题型：旧规则是库存低于固定值订货，后来可能新增考虑近期活动、销售趋势等新的计算方式。问逻辑层如何设计以更好应对变更。

- 问题：如果在排程服务中用 `if/switch(ruleType)` 判断规则，会形成控制耦合，新增规则要改旧代码，违反 OCP。
- 设计：抽象 `ReorderStrategy` 或 `ReorderRule` 接口，不同规则各自实现 `needReorder(...)` 和 `calculateQuantity(...)`。
- 排程服务只依赖接口；规则选择可由工厂、配置或规则仓储返回具体策略。
- 优点：新增“近期活动规则”只新增策略类，原有排程流程不变；符合 OCP、DIP、低耦合、高内聚。

```
interface ReorderStrategy {
    boolean needReorder(Inventory inv, SalesData sales);
    int calculateQuantity(Inventory inv, SalesData sales);
}

class FixedThresholdStrategy implements ReorderStrategy { ... }
class PromotionAwareStrategy implements ReorderStrategy { ... }
class SalesTrendStrategy implements ReorderStrategy { ... }

class ScheduleService {
    private ReorderStrategy strategy;
    PurchaseInquiry runSchedule(...) {
        if (strategy.needReorder(inv, sales)) {
            int qty = strategy.calculateQuantity(inv, sales);
            return inquiryFactory.create(product, qty);
        }
    }
}
```

例题 3：设计原则判断题

现象	违反原则/问题	修改
新增支付方式要修改 `pay(type)` 的 switch	OCP、控制耦合	抽象 PaymentStrategy，新增实现类
RechargeService 直接 new WeChatPay	DIP	依赖 ThirdPayment 接口，由工厂/DI 提供实现
User 类同时负责用户资料、发券、充值、报表导出	SRP	拆分 User、CouponService、RechargeService、ReportService
Robot 实现 Worker 但 eat/sleep 抛异常	ISP，也可能 LSP	拆成 Workable、Feedable 等小接口
Penguin 继承 Bird，但 fly() 无法满足	LSP	Bird 不承诺 fly，抽出 Flyable 接口
a.getB().getC().doSomething()	LoD/消息链	在 A 或 B 中提供委托方法，隐藏内部结构
sendEmail(Employee e) 只用 e.email	印记耦合	只传 email 或 EmailAddress

十、考场速记模板

1. 详细设计答题万能结构

- 先列类：Boundary/Controller/Service/Repository/Entity/Strategy。
- 再分职责：Controller 接收系统事件，Service 组织业务流程，信息专家承担计算，Repository 访问数据。
- 再画关系：接口实现、组合、关联；变化点用接口 + 多态。
- 再画顺序图：从用户操作到 Controller，再到 Service、Repository、策略/领域对象，最后返回 VO。
- 最后说原则：高内聚低耦合，OCP/DIP/SRP，必要时说明策略模式或状态模式。

2. 类图模板：变化点封装

```
Context/Service ----> <<interface>> Strategy
Strategy <|.. ConcreteStrategyA
Strategy <|.. ConcreteStrategyB

Context only calls Strategy.operation(...)
Adding a new behavior = add ConcreteStrategyC, not modify Context.
```

3. 顺序图模板：充值/支付/排程

```
Actor -> Page/API -> Controller -> Service
Service -> Repository: query needed PO/entity
Service -> Strategy/DomainObject: calculate/validate
Service -> Repository: save/update
Service --> Controller: ResultVO
Controller --> Page/API: show result
```

4. 设计理由模板

- 该设计把变化点抽象为接口，业务流程依赖抽象而非具体实现，符合 DIP。
- 新增一种规则/支付方式/优惠券时只需新增实现类，不修改原有流程，符合 OCP。
- 每个类只承担一个清晰职责，避免 God Class，符合 SRP 和高内聚。
- 上下文类通过组合持有策略对象，避免继承耦合，运行时可以灵活切换。